采购领域 AI Agent

整体架构设计与完整实现

Procurement Domain AI Agent — Full Architecture

包含：本体设计 · 数据库表 · RAG 构建 · Orchestrator · Sub-Agent · 工具层 · 端到端示例

1. 系统总体架构

本文档以采购领域（P2P：Procure-to-Pay）为核心场景，完整设计并实现一个结合大模型、AI Agent 与本体论的智能分析系统。系统分为五层，从下到上分别是：本体知识层、知识存储层、编排控制层、专业执行层、工具数据层。

1.1 架构五层模型

层级	名称	核心职责	关键技术
L1	本体知识层	定义业务语义与规则	OWL 2 本体 · SWRL 规则 · KPI 定义
L2	知识存储层	持久化与语义检索	Neo4j 图库 · Qdrant 向量库 · BM25
L3	Orchestrator 层	意图解析与任务编排	Claude API · DAG 调度 · 结果聚合
L4	Sub-Agent 层	专业业务分析执行	P2P Agent · 规则 Agent · 报告 Agent
L5	工具数据层	安全访问真实数据	SQL 工具 · SPARQL · 规则引擎 · ERP API

1.2 核心业务流程覆盖范围（P2P）

采购到付款（Procure-to-Pay）完整流程：

- 采购申请（Purchase Requisition / PR）— 需求部门发起
- 采购订单（Purchase Order / PO）— 采购部门审批下单
- 收货确认（Goods Receipt / GR）— 仓库确认收货
- 发票验证（Invoice Verification / IV）— 三路匹配校验
- 付款处理（Payment）— 财务执行付款

> *核心风险控制点：三路匹配（PO-GR-IV）、供应商信用、采购价格差异（PPV）、付款条款合规*

2. 采购领域本体设计

2.1 核心业务实体类（OWL Classes）

使用 OWL 2 + Turtle 格式定义采购域本体，精确建模业务概念及其继承关系：

```
# ─── 采购本体核心定义（procurement_ontology.ttl）──────────────
@prefix proc:  <http://ontology.procurement.ai/core#> .
@prefix owl:   <http://www.w3.org/2002/07/owl#> .
@prefix rdfs:  <http://www.w3.org/2000/01/rdf-schema#> .
@prefix xsd:   <http://www.w3.org/2001/XMLSchema#> .

# ── 顶级采购文档类 ──────────────────────────────────────────
proc:ProcurementDocument a owl:Class ;
  rdfs:label "采购业务单据"@zh ;
  rdfs:comment "所有采购相关单据的顶级抽象类" .

proc:PurchaseRequisition rdfs:subClassOf proc:ProcurementDocument ;
  rdfs:label "采购申请"@zh ;
  proc:hasStatus      ["PR_DRAFT","PR_PENDING","PR_APPROVED","PR_REJECTED"] ;
  proc:requiresField  proc:requestingDept, proc:requiredDate,
proc:estimatedCost .

proc:PurchaseOrder     rdfs:subClassOf proc:ProcurementDocument ;
  rdfs:label "采购订单"@zh ;
  proc:hasStatus      ["PO_OPEN","PO_PARTIAL","PO_COMPLETE","PO_CANCELLED"] ;
  proc:mustReference  proc:PurchaseRequisition ;   # PR-PO 关联约束
  proc:hasThreeWay    true .                       # 参与三路匹配

proc:GoodsReceipt      rdfs:subClassOf proc:ProcurementDocument ;
  rdfs:label "收货凭证"@zh ;
  proc:mustReference  proc:PurchaseOrder .

proc:VendorInvoice     rdfs:subClassOf proc:ProcurementDocument ;
  rdfs:label "供应商发票"@zh ;
  proc:hasStatus      ["IV_OPEN","IV_PARTIAL","IV_CLEARED","IV_BLOCKED"] ;
  proc:hasThreeWay    true .

# ── 业务伙伴类 ──────────────────────────────────────────────
proc:BusinessPartner a owl:Class ; rdfs:label "业务伙伴"@zh .
proc:Vendor rdfs:subClassOf proc:BusinessPartner ;
  rdfs:label "供应商"@zh ;
  proc:hasCreditRating  ["A","B","C","D"] ;
  proc:hasPaymentTerms  xsd:string ;        # NET30 / NET60
  proc:hasLeadTime      xsd:integer .       # 标准提前期（天）

# ── 物料类 ──────────────────────────────────────────────────
proc:Material a owl:Class ; rdfs:label "物料"@zh .
  proc:hasStandardPrice  xsd:decimal ;
  proc:hasMaterialGroup  xsd:string ;
  proc:hasUOM            xsd:string .       # 计量单位

# ── KPI 指标类 ──────────────────────────────────────────────
proc:ProcurementKPI a owl:Class .

proc:PPV a proc:ProcurementKPI ;            # 采购价格差异
  proc:formula "(ActualPrice - StdPrice) * Quantity" ;
  proc:positiveIsBad true ;
  proc:alertThreshold "5%" .

proc:POCycleTime a proc:ProcurementKPI ;    # 订单周期时间
  proc:formula "PO_Date - PR_ApprovalDate" ;
```

```
  proc:benchmarkGood  "< 3 days" ;
  proc:benchmarkBad   "> 7 days" .

proc:OnTimeDeliveryRate a proc:ProcurementKPI ;
  proc:formula "OnTimeGR / TotalGR * 100" ;
  proc:benchmarkGood  "> 95%" ;
  proc:benchmarkBad   "< 85%" .

proc:ThreeWayMatchRate a proc:ProcurementKPI ;
  proc:formula "MatchedInvoices / TotalInvoices * 100" ;
  proc:benchmarkGood  "> 98%" .
```

2.2 业务规则定义（SWRL Rules）

将隐性的采购业务规则显式化，确保 Agent 分析有规可依：

```
# ─── 采购业务规则（SWRL 自然语言描述 + 逻辑表达）──────────────

# 规则 1：三路匹配异常检测
# 条件：发票金额与收货单金额偏差 > 5%
# 触发：标记发票为 THREE_WAY_MISMATCH
PurchaseOrder(?po)
  ∧ hasInvoice(?po, ?inv)
  ∧ hasGoodsReceipt(?po, ?gr)
  ∧ invoiceAmount(?inv, ?ia)
  ∧ grAmount(?gr, ?ga)
  ∧ swrlb:abs(?diff, swrlb:subtract(?ia, ?ga))
  ∧ swrlb:greaterThan(?diff, swrlb:multiply(?ga, 0.05))
→ hasAnomaly(?inv, "THREE_WAY_MISMATCH")
  ∧ anomalySeverity(?inv, "HIGH")

# 规则 2：价格偏差预警（PPV > 5%）
PurchaseOrderLineItem(?li)
  ∧ actualUnitPrice(?li, ?ap)
  ∧ standardUnitPrice(?li, ?sp)
  ∧ swrlb:abs(?ppv, swrlb:divide(swrlb:subtract(?ap, ?sp), ?sp))
  ∧ swrlb:greaterThan(?ppv, 0.05)
→ hasPPVAlert(?li, true)
  ∧ ppvPercentage(?li, ?ppv)

# 规则 3：无 PR 直接下 PO（越权采购）
PurchaseOrder(?po)
  ∧ swrlb:not(hasRequisition(?po, ?pr))
  ∧ poAmount(?po, ?amt)
  ∧ swrlb:greaterThan(?amt, 10000)
→ hasComplianceViolation(?po, "NO_REQUISITION_BACKING")
  ∧ violationSeverity(?po, "HIGH")

# 规则 4：超审批权限
PurchaseOrder(?po)
  ∧ approvedBy(?po, ?user)
  ∧ userApprovalLimit(?user, ?limit)
  ∧ poAmount(?po, ?amt)
  ∧ swrlb:greaterThan(?amt, ?limit)
→ hasComplianceViolation(?po, "EXCEEDS_APPROVAL_AUTHORITY")

# 规则 5：供应商付款到期预警
VendorInvoice(?inv)
  ∧ invoiceDate(?inv, ?id)
  ∧ paymentTermsDays(?inv, ?days)
  ∧ swrlb:add(?dueDate, ?id, ?days)
```

```
  ∧ swrlb:lessThanOrEqual(?dueDate, swrlb:addDays(today(), 3))
  ∧ invoiceStatus(?inv, "IV_OPEN")
→ hasPaymentAlert(?inv, "DUE_SOON")
```

2.3 数据库表结构设计（PostgreSQL DDL）

ERP 采购数据的完整 Schema 设计，作为 Agent 工具层的数据基础：

2.3.1 供应商主数据表

```
-- ─── vendors（供应商主数据）──────────────────────────────────
CREATE TABLE vendors (
  vendor_id        VARCHAR(20)    PRIMARY KEY,             -- 供应商编号
  vendor_name      VARCHAR(200)   NOT NULL,
  vendor_type      VARCHAR(20)    NOT NULL                 --
MANUFACTURER/DISTRIBUTOR/SERVICE
                                  CHECK (vendor_type IN
('MANUFACTURER','DISTRIBUTOR','SERVICE')),
  credit_rating    CHAR(1)        NOT NULL DEFAULT 'B'
                                  CHECK (credit_rating IN ('A','B','C','D')),
  payment_terms    VARCHAR(20)    NOT NULL DEFAULT 'NET30', --
NET30/NET60/IMMEDIATE
  bank_account     VARCHAR(50),
  tax_number       VARCHAR(30),
  contact_name     VARCHAR(100),
  contact_email    VARCHAR(100),
  contact_phone    VARCHAR(30),
  country_code     CHAR(2)        NOT NULL DEFAULT 'CN',
  currency_code    CHAR(3)        NOT NULL DEFAULT 'CNY',
  is_active        BOOLEAN        NOT NULL DEFAULT TRUE,
  blacklisted      BOOLEAN        NOT NULL DEFAULT FALSE,
  created_at       TIMESTAMPTZ    NOT NULL DEFAULT now(),
  updated_at       TIMESTAMPTZ    NOT NULL DEFAULT now()
);

-- 供应商绩效快照（月度汇总，加速查询）
CREATE TABLE vendor_performance_monthly (
  vendor_id             VARCHAR(20)  NOT NULL REFERENCES vendors,
  year_month            CHAR(7)      NOT NULL,             -- 2025-01
  total_po_count        INT          NOT NULL DEFAULT 0,
  total_po_amount       NUMERIC(18,2)NOT NULL DEFAULT 0,
  on_time_delivery_rate NUMERIC(5,2),                       -- 0.00~100.00
  quality_pass_rate     NUMERIC(5,2),
  avg_lead_time_days    NUMERIC(6,2),
  invoice_match_rate    NUMERIC(5,2), -- 三路匹配通过率
  PRIMARY KEY (vendor_id, year_month)
);
```

2.3.2 物料主数据表

```
-- ─── materials（物料主数据）──────────────────────────────────
CREATE TABLE materials (
  material_id      VARCHAR(20)    PRIMARY KEY,
  material_name    VARCHAR(200)   NOT NULL,
  material_group   VARCHAR(20)    NOT NULL,                -- 物料大类
  material_type    VARCHAR(10)    NOT NULL                 -- RAW/SEMI/FIN/SVC
                                  CHECK (material_type IN
('RAW','SEMI','FIN','SVC')),
  uom              VARCHAR(10)    NOT NULL,                -- 计量单位
  standard_price   NUMERIC(18,4),                         -- 标准成本价
  last_po_price    NUMERIC(18,4),                         -- 最近一次采购价
  safety_stock     NUMERIC(12,3)  DEFAULT 0,
  reorder_point    NUMERIC(12,3)  DEFAULT 0,
  lead_time_days   INT            DEFAULT 7,
  is_active        BOOLEAN        DEFAULT TRUE,
  created_at       TIMESTAMPTZ    DEFAULT now()
);
```

2.3.3 采购申请表

```
-- ─── purchase_requisitions（采购申请）──────────────────────────
CREATE TABLE purchase_requisitions (
  pr_id              VARCHAR(20)   PRIMARY KEY,            -- PR编号 PR-2025-
XXXXX
  pr_date            DATE          NOT NULL DEFAULT CURRENT_DATE,
  requesting_dept    VARCHAR(50)   NOT NULL,
  requester_id       VARCHAR(20)   NOT NULL,
  required_date      DATE          NOT NULL,               -- 需求日期
  total_amount       NUMERIC(18,2) NOT NULL DEFAULT 0,
  currency_code      CHAR(3)       NOT NULL DEFAULT 'CNY',
  status             VARCHAR(20)   NOT NULL DEFAULT 'DRAFT'
                                   CHECK (status IN ('DRAFT','PENDING_APPROVAL',
                                   'APPROVED','REJECTED','PO_CREATED')),
  approver_id        VARCHAR(20),
  approval_date      TIMESTAMPTZ,
  rejection_reason   TEXT,
  notes              TEXT,
  created_at         TIMESTAMPTZ   NOT NULL DEFAULT now()
);

-- ─── pr_line_items（采购申请行项目）──────────────────────────
CREATE TABLE pr_line_items (
  pr_item_id       SERIAL         PRIMARY KEY,
  pr_id            VARCHAR(20)    NOT NULL REFERENCES purchase_requisitions,
  line_no          INT            NOT NULL,
  material_id      VARCHAR(20)    NOT NULL REFERENCES materials,
  quantity         NUMERIC(12,3)  NOT NULL CHECK (quantity > 0),
  uom              VARCHAR(10)    NOT NULL,
  estimated_price  NUMERIC(18,4),
  required_date    DATE,
  preferred_vendor VARCHAR(20)    REFERENCES vendors,
  account_code     VARCHAR(20),                           -- 费用科目
  UNIQUE (pr_id, line_no)
);
```

2.3.4 采购订单表

```
-- ─── purchase_orders（采购订单）──────────────────────────────
CREATE TABLE purchase_orders (
  po_id              VARCHAR(20)   PRIMARY KEY,            -- PO-2025-XXXXX
  pr_id              VARCHAR(20)   REFERENCES purchase_requisitions,
  po_date            DATE          NOT NULL DEFAULT CURRENT_DATE,
  vendor_id          VARCHAR(20)   NOT NULL REFERENCES vendors,
  vendor_name        VARCHAR(200)  NOT NULL,               -- 冗余存储
  company_code       VARCHAR(10)   NOT NULL DEFAULT '1000',
  plant              VARCHAR(10)   NOT NULL DEFAULT 'P001',
  total_amount       NUMERIC(18,2) NOT NULL DEFAULT 0,
  currency_code      CHAR(3)       NOT NULL DEFAULT 'CNY',
  payment_terms      VARCHAR(20)   NOT NULL,
  delivery_date      DATE,                                 -- 约定交货日期
  status             VARCHAR(20)   NOT NULL DEFAULT 'PO_OPEN'
                                   CHECK (status IN ('PO_OPEN','PO_PARTIAL',
                                   'PO_COMPLETE','PO_CANCELLED')),
  approved_by        VARCHAR(20),
  approval_amount_limit NUMERIC(18,2),                    -- 审批权限金额
  created_by         VARCHAR(20)   NOT NULL,
  created_at         TIMESTAMPTZ   NOT NULL DEFAULT now(),
  updated_at         TIMESTAMPTZ   NOT NULL DEFAULT now()
);

-- ─── po_line_items（采购订单行）──────────────────────────────
CREATE TABLE po_line_items (
```

```
  po_item_id       SERIAL          PRIMARY KEY,
  po_id            VARCHAR(20)     NOT NULL REFERENCES purchase_orders,
  pr_item_id       INT             REFERENCES pr_line_items,
  line_no          INT             NOT NULL,
  material_id      VARCHAR(20)     NOT NULL REFERENCES materials,
  material_name    VARCHAR(200)    NOT NULL,
  quantity         NUMERIC(12,3)   NOT NULL,
  uom              VARCHAR(10)     NOT NULL,
  unit_price       NUMERIC(18,4)   NOT NULL,                -- 实际采购单价
  standard_price   NUMERIC(18,4),                           -- 标准价（用于PPV计算）
  line_amount      NUMERIC(18,2)   GENERATED ALWAYS AS (quantity * unit_price)
STORED,
  delivered_qty    NUMERIC(12,3)   NOT NULL DEFAULT 0,      -- 已收货数量
  invoiced_qty     NUMERIC(12,3)   NOT NULL DEFAULT 0,      -- 已开票数量
  account_code     VARCHAR(20),
  UNIQUE (po_id, line_no)
);

-- 采购价格差异（PPV）视图
CREATE VIEW v_ppv_analysis AS
SELECT
  p.po_id, p.vendor_id, v.vendor_name,
  l.material_id, m.material_name,
  l.quantity,
  l.unit_price       AS actual_price,
  l.standard_price   AS std_price,
  l.unit_price - l.standard_price                      AS unit_ppv,
  (l.unit_price - l.standard_price) * l.quantity       AS total_ppv,
  ROUND((l.unit_price - l.standard_price)
        / NULLIF(l.standard_price, 0) * 100, 2)        AS ppv_pct,
  p.po_date
FROM po_line_items l
JOIN purchase_orders p  ON l.po_id = p.po_id
JOIN vendors v          ON p.vendor_id = v.vendor_id
JOIN materials m        ON l.material_id = m.material_id
WHERE l.standard_price IS NOT NULL;
```

2.3.5 收货与发票表

```
-- ─── goods_receipts（收货凭证）──────────────────────────────
CREATE TABLE goods_receipts (
  gr_id              VARCHAR(20)   PRIMARY KEY,             -- GR-2025-XXXXX
  po_id              VARCHAR(20)   NOT NULL REFERENCES purchase_orders,
  gr_date            DATE          NOT NULL DEFAULT CURRENT_DATE,
  warehouse_id       VARCHAR(20)   NOT NULL,
  received_by        VARCHAR(20)   NOT NULL,
  total_amount       NUMERIC(18,2) NOT NULL DEFAULT 0,
  delivery_note_no   VARCHAR(50),                           -- 送货单号
  quality_check_pass BOOLEAN       DEFAULT TRUE,
  notes              TEXT,
  created_at         TIMESTAMPTZ   NOT NULL DEFAULT now()
);

CREATE TABLE gr_line_items (
  gr_item_id       SERIAL          PRIMARY KEY,
  gr_id            VARCHAR(20)     NOT NULL REFERENCES goods_receipts,
  po_item_id       INT             NOT NULL REFERENCES po_line_items,
  line_no          INT             NOT NULL,
  received_qty     NUMERIC(12,3)   NOT NULL CHECK (received_qty > 0),
  uom              VARCHAR(10)     NOT NULL,
  unit_price       NUMERIC(18,4)   NOT NULL,
  line_amount      NUMERIC(18,2)   GENERATED ALWAYS AS (received_qty * unit_price)
STORED,
```

```
  batch_no         VARCHAR(50),
  expiry_date      DATE,
  UNIQUE (gr_id, line_no)
);

-- ─── vendor_invoices（供应商发票）──────────────────────────
CREATE TABLE vendor_invoices (
  invoice_id         VARCHAR(30)   PRIMARY KEY,            -- INV-VD-XXXXX
  po_id              VARCHAR(20)   NOT NULL REFERENCES purchase_orders,
  vendor_id          VARCHAR(20)   NOT NULL REFERENCES vendors,
  invoice_no         VARCHAR(50)   NOT NULL,               -- 供应商原始发票号
  invoice_date       DATE          NOT NULL,
  due_date           DATE          NOT NULL,               -- 到期日
  invoice_amount     NUMERIC(18,2) NOT NULL,
  tax_amount         NUMERIC(18,2) NOT NULL DEFAULT 0,
  net_amount         NUMERIC(18,2) GENERATED ALWAYS AS (invoice_amount -
tax_amount) STORED,
  currency_code      CHAR(3)       NOT NULL DEFAULT 'CNY',
  status             VARCHAR(20)   NOT NULL DEFAULT 'IV_OPEN'
                                   CHECK (status IN ('IV_OPEN','IV_PARTIAL',
                                   'IV_CLEARED','IV_BLOCKED')),
  three_way_status   VARCHAR(20)   DEFAULT 'PENDING'
                                   CHECK (three_way_status IN
('PENDING','MATCHED',
                                   'MISMATCH_QTY','MISMATCH_PRICE','BLOCKED')),
  mismatch_reason    TEXT,                                 -- 不匹配原因
  block_reason       TEXT,
  payment_id         VARCHAR(20),
  created_at         TIMESTAMPTZ   NOT NULL DEFAULT now()
);

-- 三路匹配结果视图（核心视图）
CREATE VIEW v_three_way_match AS
SELECT
  po.po_id,
  po.vendor_id,
  po.total_amount                          AS po_amount,
  COALESCE(SUM(gr_l.line_amount), 0)     AS gr_amount,
  COALESCE(SUM(inv.invoice_amount), 0)   AS invoice_amount,
  CASE
    WHEN ABS(COALESCE(SUM(inv.invoice_amount),0)
             - COALESCE(SUM(gr_l.line_amount),0))
       / NULLIF(COALESCE(SUM(gr_l.line_amount),0), 0) > 0.05
    THEN 'MISMATCH'
    ELSE 'MATCHED'
  END                                      AS match_status,
  ROUND(
    (COALESCE(SUM(inv.invoice_amount),0)
     - COALESCE(SUM(gr_l.line_amount),0)) /
    NULLIF(COALESCE(SUM(gr_l.line_amount),0),0) * 100, 2
  )                                        AS variance_pct
FROM purchase_orders po
LEFT JOIN goods_receipts gr     ON gr.po_id = po.po_id
LEFT JOIN gr_line_items gr_l    ON gr_l.gr_id = gr.gr_id
LEFT JOIN vendor_invoices inv   ON inv.po_id = po.po_id
GROUP BY po.po_id, po.vendor_id, po.total_amount;

-- ─── payments（付款记录）──────────────────────────────────
CREATE TABLE payments (
  payment_id       VARCHAR(20)    PRIMARY KEY,
  invoice_id       VARCHAR(30)    NOT NULL REFERENCES vendor_invoices,
  vendor_id        VARCHAR(20)    NOT NULL REFERENCES vendors,
  payment_date     DATE           NOT NULL,
```

```
  payment_amount   NUMERIC(18,2)  NOT NULL,
  payment_method   VARCHAR(20)    NOT NULL,                -- BANK_TRANSFER/CHECK
  bank_ref_no      VARCHAR(50),
  early_pay_disc   NUMERIC(18,2)  DEFAULT 0,               -- 早付折扣金额
  created_by       VARCHAR(20)    NOT NULL,
  created_at       TIMESTAMPTZ    NOT NULL DEFAULT now()
);
```

2.3.6 关键索引设计

```
-- 高频查询索引
CREATE INDEX idx_po_vendor       ON purchase_orders(vendor_id, status, po_date);
CREATE INDEX idx_po_status       ON purchase_orders(status, created_at DESC);
CREATE INDEX idx_gr_po           ON goods_receipts(po_id, gr_date);
CREATE INDEX idx_inv_po          ON vendor_invoices(po_id, status);
CREATE INDEX idx_inv_due         ON vendor_invoices(due_date, status)
                                  WHERE status = 'IV_OPEN';
CREATE INDEX idx_inv_vendor      ON vendor_invoices(vendor_id, status);
CREATE INDEX idx_ppv_po_date     ON po_line_items(po_id)
  INCLUDE (unit_price, standard_price, quantity);
```

3. 知识图谱与 RAG 构建

3.1 Neo4j 图数据库 Schema

将本体实例化为图数据，支持多跳关系查询（如：供应商→PO→GR→发票路径追踪）：

```
// ─── Neo4j 节点与关系定义（Cypher）──────────────────────────

// 约束与索引（优先创建）
CREATE CONSTRAINT vendor_id_unique FOR (v:Vendor) REQUIRE v.vendor_id IS UNIQUE;
CREATE CONSTRAINT po_id_unique     FOR (p:PurchaseOrder) REQUIRE p.po_id IS
UNIQUE;
CREATE CONSTRAINT inv_id_unique    FOR (i:Invoice) REQUIRE i.invoice_id IS
UNIQUE;
CREATE INDEX material_idx          FOR (m:Material) ON (m.material_id);

// ── 创建供应商节点 ──────────────────────────────────────────
MERGE (v:Vendor {vendor_id: 'V-001'})
SET v += {
  name:           '上海精密零件有限公司',
  credit_rating: 'A',
  payment_terms: 'NET30',
  lead_time:     7,
  is_active:     true,
  ontology_class:'proc:Vendor'
}

// ── 创建采购订单节点 ────────────────────────────────────────
MERGE (po:PurchaseOrder {po_id: 'PO-2025-00831'})
SET po += {
  po_date:     date('2025-02-10'),
  amount:      485000.00,
  currency:    'CNY',
  status:      'PO_PARTIAL',
  ontology_class:'proc:PurchaseOrder'
}

// ── 建立核心业务关系 ────────────────────────────────────────
MATCH (po:PurchaseOrder {po_id:'PO-2025-00831'}),
      (v:Vendor {vendor_id:'V-001'})
CREATE (po)-[:ISSUED_TO {
  payment_terms: 'NET30',
  delivery_date: date('2025-02-25')
}]->(v)

// ── 收货节点关联 ────────────────────────────────────────────
MERGE (gr:GoodsReceipt {gr_id:'GR-2025-00456'})
SET gr += { gr_date: date('2025-02-22'), amount: 482000.00 }
MATCH (po:PurchaseOrder {po_id:'PO-2025-00831'}),
      (gr:GoodsReceipt {gr_id:'GR-2025-00456'})
CREATE (gr)-[:REFERENCES]->(po)

// ── 发票节点关联（三路匹配核心路径）──────────────────────
MERGE (inv:Invoice {invoice_id:'INV-V-2025-08831'})
SET inv += {
  invoice_date:   date('2025-02-28'),
  due_date:       date('2025-03-30'),
  amount:         510000.00,            -- 比收货单多 28000（5.8%，触发异常）
  status:         'IV_BLOCKED',
  three_way:      'MISMATCH_PRICE'
}
MATCH (inv:Invoice {invoice_id:'INV-V-2025-08831'}),
```

```
      (po:PurchaseOrder {po_id:'PO-2025-00831'})
CREATE (inv)-[:REFERENCES]->(po)

// ── 多跳查询：找出所有三路匹配异常及其供应商 ──────────────
MATCH path = (v:Vendor)<-[:ISSUED_TO]-(po:PurchaseOrder)
             <-[:REFERENCES]-(inv:Invoice)
WHERE inv.three_way = 'MISMATCH_PRICE'
  AND inv.status = 'IV_BLOCKED'
RETURN v.name         AS vendor,
       po.po_id       AS po_id,
       po.amount      AS po_amount,
       inv.invoice_id AS invoice_id,
       inv.amount     AS invoice_amount,
       inv.amount - po.amount AS variance
ORDER BY ABS(inv.amount - po.amount) DESC
```

3.2 RAG 索引构建（Python 实现）

将本体知识向量化，支持 Agent 按语义检索相关业务知识：

```
# ─── rag_builder.py ──────────────────────────────────────────
"""
采购本体 RAG 索引构建器
支持双路检索：稠密向量（BGE-M3）+ BM25 稀疏索引
"""

from qdrant_client import QdrantClient
from qdrant_client.models import (
    VectorParams, Distance, SparseVectorParams,
    PointStruct, SparseVector
)
from sentence_transformers import SentenceTransformer
from transformers import AutoTokenizer
import json, uuid
from dataclasses import dataclass
from typing import Optional

@dataclass
class OntologyDocument:
    doc_id:   str
    doc_type: str       # ontology_class / kpi / business_rule / process
    module:   str       # PROCUREMENT
    text:     str       # 自然语言描述（用于向量化）
    metadata: dict

class ProcurementRAGBuilder:
    def __init__(self):
        self.client     = QdrantClient(host="localhost", port=6333)
        self.dense_model= SentenceTransformer("BAAI/bge-m3")
        self.collection = "procurement_ontology"
        self._init_collection()

    def _init_collection(self):
        """初始化 Qdrant 集合（稠密+稀疏双路）"""
        self.client.recreate_collection(
            collection_name=self.collection,
            vectors_config={
                "dense": VectorParams(size=1024, distance=Distance.COSINE)
            },
            sparse_vectors_config={
                "sparse": SparseVectorParams()
```

```
            }
        )
        print(f"集合 {self.collection} 初始化完成")

    def _bm25_encode(self, text: str) -> SparseVector:
        """简化 BM25 稀疏编码（生产建议用 Qdrant FastEmbed）"""
        from sklearn.feature_extraction.text import TfidfVectorizer
        vec = TfidfVectorizer(max_features=512)
        tfidf = vec.fit_transform([text])
        indices = tfidf.nonzero()[1].tolist()
        values  = tfidf.data.tolist()
        return SparseVector(indices=indices, values=values)

    def index_ontology_classes(self):
        """将本体类定义向量化入库"""
        classes = [
            OntologyDocument(
                doc_id="cls_purchase_order",
                doc_type="ontology_class",
                module="PROCUREMENT",
                text="""
业务实体：采购订单（Purchase Order / PO）
定义：采购方与供应商之间的正式合同文件，是采购业务的核心凭证。
关键属性：PO 编号、供应商、总金额、付款条款、交货日期、状态
关联实体：采购申请（PR）、收货凭证（GR）、供应商发票（IV）
业务约束：金额超 1 万元必须有对应采购申请；金额超 10 万需总监审批
状态流转：PO_OPEN → PO_PARTIAL → PO_COMPLETE
参与三路匹配：是（PO-GR-IV 金额和数量必须匹配，偏差阈值 5%）
适用场景：采购合规分析、三路匹配验证、供应商管理
                """.strip(),
                metadata={"entity": "PurchaseOrder", "process": "P2P"}
            ),
            OntologyDocument(
                doc_id="cls_vendor_invoice",
                doc_type="ontology_class",
                module="PROCUREMENT",
                text="""
业务实体：供应商发票（Vendor Invoice）
定义：供应商提供的付款请求凭证，需经三路匹配验证后方可付款。
关键属性：发票号、发票日期、到期日、发票金额、税额、三路匹配状态
关联实体：采购订单（PO）、收货凭证（GR）
风险标志：IV_BLOCKED（三路不匹配/合规问题）
付款前提：三路匹配状态必须为 MATCHED；无合规违规
预警规则：到期日前 3 天触发付款提醒
适用场景：付款合规审核、应付账款分析
                """.strip(),
                metadata={"entity": "VendorInvoice", "process": "P2P"}
            ),
            OntologyDocument(
                doc_id="kpi_three_way_match",
                doc_type="kpi",
                module="PROCUREMENT",
                text="""
KPI 名称：三路匹配通过率（Three-Way Match Rate）
计算公式：已匹配发票数 / 总发票数 × 100%
数据来源：vendor_invoices.three_way_status
行业基准：良好 > 98%；预警 < 95%；差 < 90%
核查维度：金额匹配（偏差≤5%）、数量匹配、单价匹配
失败原因分类：MISMATCH_QTY（数量差异）、MISMATCH_PRICE（价格差异）
改善方向：优化供应商发票管理、加强 PO 准确性
适用分析场景：采购合规分析、供应商付款审核、内控评估
                """.strip(),
                metadata={"kpi": "ThreeWayMatchRate"}
```

```
            ),
            OntologyDocument(
                doc_id="kpi_ppv",
                doc_type="kpi",
                module="PROCUREMENT",
                text="""
KPI 名称：采购价格差异（Purchase Price Variance / PPV）
计算公式：(实际采购单价 - 标准成本价) × 采购数量
正负含义：负值表示节约成本（实际价低于标准价），正值表示超支
告警阈值：单品种 PPV% > 5% 触发预警
汇总维度：按供应商/物料大类/采购员/时间段汇总
数据来源：po_line_items.unit_price vs materials.standard_price
适用分析场景：成本控制分析、供应商谈判效果评估、预算执行分析
                """.strip(),
                metadata={"kpi": "PPV"}
            ),
        ]

        points = []
        for doc in classes:
            dense_vec = self.dense_model.encode(
                doc.text, normalize_embeddings=True
            ).tolist()
            sparse_vec = self._bm25_encode(doc.text)

            points.append(PointStruct(
                id=str(uuid.uuid5(uuid.NAMESPACE_DNS, doc.doc_id)),
                vector={"dense": dense_vec, "sparse": sparse_vec},
                payload={
                    "doc_id":   doc.doc_id,
                    "doc_type": doc.doc_type,
                    "module":   doc.module,
                    "text":     doc.text,
                    **doc.metadata
                }
            ))

        self.client.upsert(
            collection_name=self.collection,
            points=points
        )
        print(f"已索引 {len(points)} 条本体知识")

    def retrieve(self,
                 query: str,
                 doc_types: Optional[list] = None,
                 top_k: int = 5) -> list[dict]:
        """
        混合检索：稠密向量 + BM25 稀疏，结果 RRF 融合
        """
        from qdrant_client.models import (
            SearchRequest, NamedVector, NamedSparseVector,
            Filter, FieldCondition, MatchAny
        )

        query_filter = None
        if doc_types:
            query_filter = Filter(must=[
                FieldCondition(key="doc_type", match=MatchAny(any=doc_types))
            ])

        dense_vec = self.dense_model.encode(
            query, normalize_embeddings=True
```

```
        ).tolist()

        # 稠密检索
        dense_results = self.client.search(
            collection_name=self.collection,
            query_vector=NamedVector(name="dense", vector=dense_vec),
            query_filter=query_filter,
            limit=top_k * 2
        )

        # 稀疏检索
        sparse_vec = self._bm25_encode(query)
        sparse_results = self.client.search(
            collection_name=self.collection,
            query_vector=NamedSparseVector(
                name="sparse",
                vector=sparse_vec
            ),
            query_filter=query_filter,
            limit=top_k * 2
        )

        # RRF 分数融合
        scores = {}
        for rank, r in enumerate(dense_results):
            scores[r.id] = scores.get(r.id, 0) + 1 / (60 + rank + 1)
        for rank, r in enumerate(sparse_results):
            scores[r.id] = scores.get(r.id, 0) + 1 / (60 + rank + 1)

        # 按 RRF 分数排序，取 top_k
        all_results = {r.id: r for r in dense_results + sparse_results}
        sorted_ids  = sorted(scores, key=scores.get, reverse=True)[:top_k]

        return [all_results[rid].payload for rid in sorted_ids if rid in
all_results]
```

4. Orchestrator 编排层设计

4.1 意图解析与场景路由

```
# ─── orchestrator.py ──────────────────────────────────────────
"""
采购 Agent Orchestrator
负责：意图解析 → DAG 规划 → Sub-Agent 调度 → 结果聚合
"""
import asyncio, json
from dataclasses import dataclass, field
from typing import Any
from anthropic import Anthropic

client = Anthropic()

INTENT_ROUTING_PROMPT = """
你是采购 AI 分析系统的路由引擎。
分析用户请求，输出 JSON（不要有其他内容）：

{
  "scenario": "THREE_WAY_MATCH | PPV_ANALYSIS | VENDOR_PERFORMANCE |
               PAYMENT_COMPLIANCE | PO_CYCLE_TIME | SPEND_ANALYSIS",
  "time_range": {"start": "YYYY-MM-DD", "end": "YYYY-MM-DD"},
  "filters": {"vendor_id": null, "material_group": null, "company_code": "1000"},
  "analysis_depth": "SUMMARY | DRILLDOWN | ROOT_CAUSE",
  "required_agents": ["p2p_agent", "rule_agent", "report_agent"],
  "priority_kpis": ["三路匹配率", "PPV", "准时交货率"]
}

今天日期：{today}
用户请求：{request}
"""

@dataclass
class SubTaskResult:
    task_name: str
    status:    str         # success / failed / partial
    data:      Any
    agent:     str
    duration:  float       # 秒

@dataclass
class AnalysisReport:
    scenario:   str
    findings:   list[dict]
    kpis:       dict
    anomalies:  list[dict]
    actions:    list[dict]
    raw_results:dict = field(default_factory=dict)

async def run_orchestrator(user_request: str) -> AnalysisReport:
    """Orchestrator 入口函数"""
    from datetime import date

    # Step 1: 从本体 RAG 检索相关知识
    rag = ProcurementRAGBuilder()
    ontology_context = rag.retrieve(user_request, top_k=6)
    print(f"[Orchestrator] 检索到 {len(ontology_context)} 条本体知识")

    # Step 2: 解析用户意图
    intent_response = client.messages.create(
```

```
        model="claude-sonnet-4-6",
        max_tokens=800,
        messages=[{
            "role": "user",
            "content": INTENT_ROUTING_PROMPT.format(
                today=str(date.today()),
                request=user_request
            )
        }]
    )
    intent = json.loads(intent_response.content[0].text)
    print(f"[Orchestrator] 分析场景: {intent['scenario']}")

    # Step 3: 构建 DAG 执行计划
    dag = build_dag(intent, ontology_context)

    # Step 4: 并行执行 Sub-Agent
    results = await execute_dag(dag, intent)

    # Step 5: 聚合结果生成报告
    return await aggregate_results(results, intent, ontology_context)
```

4.2 DAG 任务规划与并行调度

```
@dataclass
class DAGTask:
    name:        str
    agent_func:  callable
    depends_on:  list[str]   # 依赖的前置任务名
    inputs:      dict        # 静态输入参数
    priority:    int = 0     # 数字越大优先级越高

def build_dag(intent: dict, ontology_ctx: list) -> list[DAGTask]:
    """根据分析意图构建 DAG，三路匹配场景示例"""
    base_filters = intent.get("filters", {})
    time_range   = intent.get("time_range", {})

    if intent["scenario"] == "THREE_WAY_MATCH":
        return [
            # ── 第一层: 数据采集（全部并行）──────────────────
            DAGTask(
                name="fetch_pos",
                agent_func=fetch_purchase_orders,
                depends_on=[],
                inputs={"filters": base_filters, "time_range": time_range}
            ),
            DAGTask(
                name="fetch_gr",
                agent_func=fetch_goods_receipts,
                depends_on=[],
                inputs={"filters": base_filters, "time_range": time_range}
            ),
            DAGTask(
                name="fetch_invoices",
                agent_func=fetch_vendor_invoices,
                depends_on=[],
                inputs={"filters": base_filters, "time_range": time_range}
            ),
            DAGTask(
                name="load_ontology",
                agent_func=load_matching_rules,
                depends_on=[],
                inputs={"ontology_ctx": ontology_ctx}
```

```
            ),

            # ── 第二层：三路匹配分析（依赖第一层全部完成）──────
            DAGTask(
                name="three_way_match",
                agent_func=run_p2p_agent_three_way,
                depends_on=["fetch_pos", "fetch_gr", "fetch_invoices"],
                inputs={}
            ),
            DAGTask(
                name="rule_validation",
                agent_func=run_rule_agent,
                depends_on=["fetch_pos", "fetch_invoices", "load_ontology"],
                inputs={}
            ),

            # ── 第三层：报告生成（依赖第二层）──────────────────
            DAGTask(
                name="generate_report",
                agent_func=run_report_agent,
                depends_on=["three_way_match", "rule_validation"],
                inputs={"scenario": "THREE_WAY_MATCH"}
            )
        ]
    # 可扩展其他场景...
    raise ValueError(f"未知场景: {intent['scenario']}")

async def execute_dag(dag: list[DAGTask], intent: dict) -> dict[str,
SubTaskResult]:
    """拓扑排序 + asyncio 并行执行 DAG"""
    completed = {}   # {task_name: SubTaskResult}
    pending   = {t.name: t for t in dag}

    while pending:
        # 找出所有依赖已满足的任务
        ready = [
            t for t in pending.values()
            if all(dep in completed for dep in t.depends_on)
        ]
        if not ready:
            raise RuntimeError("DAG 死锁：存在循环依赖")

        print(f"[DAG] 并行执行 {len(ready)} 个任务: "
              f"{[t.name for t in ready]}")

        # 并行执行这批任务
        import time
        async def run_task(task: DAGTask):
            start = time.time()
            # 合并依赖任务的结果作为输入
            dep_outputs = {
                dep: completed[dep].data
                for dep in task.depends_on
            }
            merged_inputs = {**task.inputs, "dep_outputs": dep_outputs}
            result = await task.agent_func(**merged_inputs)
            return SubTaskResult(
                task_name=task.name,
                status="success",
                data=result,
                agent=task.agent_func.__name__,
                duration=round(time.time() - start, 2)
            )
```

```
        batch_results = await asyncio.gather(
            *[run_task(t) for t in ready],
            return_exceptions=True
        )

        for task, result in zip(ready, batch_results):
            if isinstance(result, Exception):
                completed[task.name] = SubTaskResult(
                    task_name=task.name,
                    status="failed",
                    data={"error": str(result)},
                    agent=task.agent_func.__name__,
                    duration=0
                )
            else:
                completed[task.name] = result
            del pending[task.name]

    return completed
```

5. 采购 Sub-Agent（P2P Agent）实现

5.1 System Prompt 设计

```
P2P_AGENT_SYSTEM_PROMPT = """
你是企业 ERP 系统的采购业务分析专家（P2P Agent）。

## 你的专业领域
- 采购到付款（P2P）全流程分析
- 三路匹配验证（PO-GR-Invoice）
- 采购价格差异（PPV）分析
- 供应商绩效评估
- 付款合规性审核

## 分析规范
1. 所有结论必须基于工具返回的真实数据，不得推测或假设
2. 所有金额统一使用人民币（CNY），精确到分
3. 百分比保留两位小数
4. 每条发现必须包含：现象描述、数据依据、风险等级（HIGH/MEDIUM/LOW）、改善建议
5. 引用本体知识中的 KPI 基准判断异常

## 本体知识上下文
{ontology_context}

## 可用工具
- query_pos：查询采购订单列表
- query_gr：查询收货凭证
- query_invoices：查询供应商发票
- run_three_way_match：执行三路匹配计算
- calculate_ppv：计算采购价格差异
- validate_compliance：合规规则校验
- get_vendor_scorecard：获取供应商评分卡

## 输出格式
最终分析结果必须是结构化 JSON：
{
  "summary": "一句话总结",
  "findings": [{"title","description","severity","data","recommendation"}],
  "kpis": {"three_way_match_rate","ppv_total","on_time_rate"},
  "anomalies": [{"type","affected_docs","severity","detail"}],
  "action_items": [{"priority","action","owner","deadline"}]
}
"""

async def run_p2p_agent_three_way(dep_outputs: dict, **kwargs) -> dict:
    """
    P2P Agent 执行三路匹配分析
    dep_outputs: 来自前置任务（fetch_pos/fetch_gr/fetch_invoices）的数据
    """
    rag = ProcurementRAGBuilder()
    ontology_ctx = rag.retrieve(
        "三路匹配 发票异常 采购合规 PPV",
        doc_types=["kpi", "business_rule", "ontology_class"],
        top_k=5
    )

    system_prompt = P2P_AGENT_SYSTEM_PROMPT.format(
        ontology_context=format_ontology_context(ontology_ctx)
    )

    # 构建工具集
    tools = build_p2p_tools()
```

```
    messages = [{
        "role": "user",
        "content": f"""
请对以下采购数据执行完整的三路匹配分析:

采购订单数据摘要:
{json.dumps(dep_outputs.get('fetch_pos', {})[:5], ensure_ascii=False, indent=2)}

收货凭证数据摘要:
{json.dumps(dep_outputs.get('fetch_gr', {})[:5], ensure_ascii=False, indent=2)}

供应商发票数据摘要:
{json.dumps(dep_outputs.get('fetch_invoices', {})[:5], ensure_ascii=False,
indent=2)}

请:
1. 执行三路匹配验证,计算匹配率
2. 识别所有不匹配的发票(金额或数量偏差 > 5%)
3. 计算 PPV 并识别价格异常
4. 检查付款合规性
5. 生成结构化分析报告
"""
    }]

    # Agentic Loop
    max_iterations = 12
    for i in range(max_iterations):
        response = client.messages.create(
            model="claude-sonnet-4-6",
            max_tokens=4096,
            system=system_prompt,
            tools=tools,
            messages=messages
        )

        if response.stop_reason == "end_turn":
            # 提取 JSON 结果
            final_text = next(
                (b.text for b in response.content if hasattr(b,'text')), ""
            )
            return extract_json_result(final_text)

        if response.stop_reason == "tool_use":
            messages.append({"role": "assistant", "content": response.content})
            tool_results = []
            for block in response.content:
                if block.type == "tool_use":
                    result = await execute_tool(block.name, block.input)
                    tool_results.append({
                        "type": "tool_result",
                        "tool_use_id": block.id,
                        "content": json.dumps(result, ensure_ascii=False)
                    })
            messages.append({"role": "user", "content": tool_results})

    raise TimeoutError(f"P2P Agent 超过最大迭代次数 {max_iterations}")
```

5.2 工具定义（Tool Schema）

```
def build_p2p_tools() -> list[dict]:
```

```
    """构建 P2P Agent 可用的工具列表（Claude Tool Use 格式）"""
    return [
        {
            "name": "run_three_way_match",
            "description": """
执行三路匹配分析（PO-GR-Invoice）。
返回：每个采购订单的匹配状态、偏差金额、偏差百分比。
触发场景：需要验证发票与收货、采购订单是否一致时使用。
            """.strip(),
            "input_schema": {
                "type": "object",
                "properties": {
                    "po_ids": {
                        "type": "array", "items": {"type": "string"},
                        "description": "要检查的PO编号列表，为空则检查全部"
                    },
                    "tolerance_pct": {
                        "type": "number", "default": 5.0,
                        "description": "允许偏差百分比，默认5%"
                    },
                    "date_from": {"type": "string", "description": "YYYY-MM-DD"},
                    "date_to":   {"type": "string", "description": "YYYY-MM-DD"}
                },
                "required": []
            }
        },
        {
            "name": "calculate_ppv",
            "description": """
计算采购价格差异（PPV）。
公式：(实际单价 - 标准成本价) × 采购数量。
返回：汇总PPV金额、偏差最大的TOP10物料、按供应商/物料大类汇总。
            """.strip(),
            "input_schema": {
                "type": "object",
                "properties": {
                    "group_by": {
                        "type": "string",
                        "enum": ["vendor", "material", "material_group",
"buyer"],
                        "description": "汇总维度"
                    },
                    "min_ppv_pct": {
                        "type": "number", "default": 0,
                        "description": "只返回PPV百分比超过此值的记录"
                    },
                    "date_from": {"type": "string"},
                    "date_to":   {"type": "string"}
                },
                "required": ["group_by"]
            }
        },
        {
            "name": "validate_compliance",
            "description": """
对采购数据执行合规规则校验，包括：
- 无PR直接下PO（越权采购）
- 审批权限超限
- 价格偏差超阈值未走审批
- 供应商黑名单检查
返回：违规清单，每条包含违规类型、严重程度、涉及单据。
            """.strip(),
            "input_schema": {
```

```
                "type": "object",
                "properties": {
                    "po_ids": {
                        "type": "array", "items": {"type": "string"},
                        "description": "要检查的PO列表，为空则检查全部"
                    },
                    "rule_types": {
                        "type": "array",
                        "items": {
                            "type": "string",
                            "enum": ["NO_PR_BACKING","APPROVAL_LIMIT",
                                     "BLACKLISTED_VENDOR","PRICE_DEVIATION"]
                        },
                        "description": "要检查的规则类型，为空则检查全部"
                    }
                },
                "required": []
            }
        },
        {
            "name": "get_vendor_scorecard",
            "description": """
获取供应商综合评分卡，包含:
准时交货率、质量合格率、发票匹配率、价格稳定性、平均提前期。
            """.strip(),
            "input_schema": {
                "type": "object",
                "properties": {
                    "vendor_ids": {
                        "type": "array", "items": {"type": "string"}
                    },
                    "period_months": {
                        "type": "integer", "default": 3,
                        "description": "统计最近N个月"
                    }
                },
                "required": []
            }
        }
    ]
```

6. 工具层完整实现

6.1 SQL 查询工具（参数化 + 防注入）

```
# ─── tools/sql_tools.py ──────────────────────────────────────
"""
采购数据查询工具层
安全原则：不接受自由 SQL，只接受结构化意图+参数
"""
import asyncpg, json
from typing import Optional
from datetime import date

# 数据库连接池（全局复用）
_pool: asyncpg.Pool = None

async def get_pool() -> asyncpg.Pool:
    global _pool
    if _pool is None:
        _pool = await asyncpg.create_pool(
            dsn="postgresql://erp_agent:pass@localhost/erp_db",
            min_size=2, max_size=10
        )
    return _pool

async def execute_tool(tool_name: str, params: dict) -> dict:
    """工具分发器，Agent 调用的统一入口"""
    handlers = {
        "run_three_way_match": tool_three_way_match,
        "calculate_ppv":       tool_calculate_ppv,
        "validate_compliance": tool_validate_compliance,
        "get_vendor_scorecard":tool_vendor_scorecard,
        "query_pos":           tool_query_pos,
        "query_invoices":      tool_query_invoices,
    }
    handler = handlers.get(tool_name)
    if not handler:
        return {"error": f"未知工具: {tool_name}"}
    try:
        return await handler(**params)
    except Exception as e:
        return {"error": str(e), "tool": tool_name}

async def tool_three_way_match(
    po_ids: list = None,
    tolerance_pct: float = 5.0,
    date_from: str = None,
    date_to: str   = None
) -> dict:
    """三路匹配分析工具"""
    pool = await get_pool()

    # 基础三路匹配查询（参数化，防注入）
    query = """
        SELECT
            po.po_id,
            po.vendor_id,
            v.vendor_name,
            po.total_amount                                    AS po_amount,
            COALESCE(SUM(DISTINCT gr.total_amount), 0)       AS gr_amount,
            COALESCE(SUM(DISTINCT inv.invoice_amount), 0)    AS inv_amount,
            COUNT(DISTINCT gr.gr_id)                           AS gr_count,
```

```
            COUNT(DISTINCT inv.invoice_id)                    AS inv_count,
            CASE
              WHEN COALESCE(SUM(DISTINCT gr.total_amount),0) = 0
                THEN 'NO_GR'
              WHEN COALESCE(SUM(DISTINCT inv.invoice_amount),0) = 0
                THEN 'NO_INVOICE'
              WHEN ABS(COALESCE(SUM(DISTINCT inv.invoice_amount),0)
                       - COALESCE(SUM(DISTINCT gr.total_amount),0))
                   / NULLIF(COALESCE(SUM(DISTINCT gr.total_amount),0),0)
                   > ($1::float / 100)
              THEN 'MISMATCH'
              ELSE 'MATCHED'
            END                                               AS match_status,
            ROUND(
              (COALESCE(SUM(DISTINCT inv.invoice_amount),0)
               - COALESCE(SUM(DISTINCT gr.total_amount),0))
              / NULLIF(COALESCE(SUM(DISTINCT gr.total_amount),0),0) * 100, 2
            )                                                 AS variance_pct
        FROM purchase_orders po
        LEFT JOIN vendors v             ON v.vendor_id = po.vendor_id
        LEFT JOIN goods_receipts gr     ON gr.po_id = po.po_id
        LEFT JOIN vendor_invoices inv   ON inv.po_id = po.po_id
        WHERE ($2::varchar[] IS NULL OR po.po_id = ANY($2))
          AND ($3::date IS NULL OR po.po_date >= $3)
          AND ($4::date IS NULL OR po.po_date <= $4)
        GROUP BY po.po_id, po.vendor_id, v.vendor_name, po.total_amount
        ORDER BY ABS(
            COALESCE(SUM(DISTINCT inv.invoice_amount),0)
            - COALESCE(SUM(DISTINCT gr.total_amount),0)
        ) DESC
        LIMIT 200
    """
    async with pool.acquire() as conn:
        rows = await conn.fetch(
            query,
            tolerance_pct,
            po_ids or None,
            date.fromisoformat(date_from) if date_from else None,
            date.fromisoformat(date_to)   if date_to   else None
        )

    records   = [dict(r) for r in rows]
    matched   = sum(1 for r in records if r['match_status'] == 'MATCHED')
    mismatched= sum(1 for r in records if r['match_status'] == 'MISMATCH')
    total     = len(records)

    return {
        "summary": {
            "total_pos":        total,
            "matched":          matched,
            "mismatched":       mismatched,
            "match_rate_pct":   round(matched / total * 100, 2) if total else 0,
            "total_variance":   sum(
                abs(r['inv_amount'] - r['gr_amount'])
                for r in records if r['match_status'] == 'MISMATCH'
            )
        },
        "mismatches": [r for r in records if r['match_status'] != 'MATCHED'],
        "all_records": records[:50]   # 截断，避免上下文溢出
    }

async def tool_calculate_ppv(
    group_by: str,
```

```
    min_ppv_pct: float = 0,
    date_from: str = None,
    date_to:   str = None
) -> dict:
    """计算采购价格差异 PPV"""
    VALID_GROUP_BY = {"vendor","material","material_group","buyer"}
    if group_by not in VALID_GROUP_BY:
        return {"error": f"group_by 必须是 {VALID_GROUP_BY} 之一"}

    # 根据 group_by 动态选择分组字段（白名单控制，防注入）
    group_fields = {
        "vendor":         "po.vendor_id, v.vendor_name",
        "material":       "l.material_id, m.material_name",
        "material_group": "m.material_group",
        "buyer":          "po.created_by AS buyer_id"
    }

    pool = await get_pool()
    query = f"""
        SELECT
            {group_fields[group_by]},
            COUNT(DISTINCT po.po_id)              AS po_count,
            SUM(l.quantity)                       AS total_qty,
            SUM(l.line_amount)                    AS total_actual,
            SUM(l.standard_price * l.quantity)    AS total_standard,
            SUM((l.unit_price - l.standard_price)
                 * l.quantity)                    AS total_ppv,
            ROUND(
                SUM((l.unit_price - l.standard_price) * l.quantity)
                / NULLIF(SUM(l.standard_price * l.quantity), 0) * 100, 2
            )                                     AS ppv_pct
        FROM po_line_items l
        JOIN purchase_orders po ON l.po_id = po.po_id
        JOIN vendors v          ON po.vendor_id = v.vendor_id
        JOIN materials m        ON l.material_id = m.material_id
        WHERE l.standard_price IS NOT NULL
          AND ($1::date IS NULL OR po.po_date >= $1)
          AND ($2::date IS NULL OR po.po_date <= $2)
        GROUP BY {group_fields[group_by]}
        HAVING ABS(ROUND(
            SUM((l.unit_price - l.standard_price) * l.quantity)
            / NULLIF(SUM(l.standard_price * l.quantity), 0) * 100, 2
        )) >= $3
        ORDER BY ABS(SUM((l.unit_price - l.standard_price) * l.quantity)) DESC
        LIMIT 50
    """
    async with pool.acquire() as conn:
        rows = await conn.fetch(
            query,
            date.fromisoformat(date_from) if date_from else None,
            date.fromisoformat(date_to)   if date_to   else None,
            min_ppv_pct
        )
    records = [dict(r) for r in rows]

    return {
        "group_by":    group_by,
        "total_ppv":   sum(r.get('total_ppv',0) for r in records),
        "records":     records,
        "top5_worst":  sorted(records, key=lambda x: abs(x.get('ppv_pct',0)),
                              reverse=True)[:5]
    }
```

6.2 合规规则验证工具

```
async def tool_validate_compliance(
    po_ids:     list = None,
    rule_types: list = None
) -> dict:
    """
    合规规则校验工具
    规则来自本体定义，在 SQL 层实现确定性计算
    """
    pool = await get_pool()
    violations = []

    # ── 规则 1：无 PR 直接下 PO（越权采购）──────────────────────
    if not rule_types or "NO_PR_BACKING" in rule_types:
        async with pool.acquire() as conn:
            rows = await conn.fetch("""
                SELECT po.po_id, po.total_amount,
                       po.vendor_id, po.created_by, po.po_date
                FROM purchase_orders po
                WHERE po.pr_id IS NULL
                  AND po.total_amount > 10000          -- 阈值来自本体规则
                  AND po.status != 'PO_CANCELLED'
                  AND ($1::varchar[] IS NULL OR po.po_id = ANY($1))
            """, po_ids)
            for r in rows:
                violations.append({
                    "rule_id":     "PROC-RULE-001",
                    "rule_type":   "NO_PR_BACKING",
                    "severity":    "HIGH",
                    "po_id":       r['po_id'],
                    "amount":      float(r['total_amount']),
                    "created_by":  r['created_by'],
                    "description": f"PO {r['po_id']} 金额 ¥
{r['total_amount']:,.2f}"
                                   " 无对应采购申请（越权采购）",
                    "ontology_ref": "proc:NoPRBacking"
                })

    # ── 规则 2：超审批权限 ──────────────────────────────────────
    if not rule_types or "APPROVAL_LIMIT" in rule_types:
        async with pool.acquire() as conn:
            rows = await conn.fetch("""
                SELECT po.po_id, po.total_amount,
                       po.approved_by, po.approval_amount_limit
                FROM purchase_orders po
                WHERE po.approved_by IS NOT NULL
                  AND po.approval_amount_limit IS NOT NULL
                  AND po.total_amount > po.approval_amount_limit
                  AND ($1::varchar[] IS NULL OR po.po_id = ANY($1))
            """, po_ids)
            for r in rows:
                violations.append({
                    "rule_id":     "PROC-RULE-002",
                    "rule_type":   "APPROVAL_LIMIT",
                    "severity":    "HIGH",
                    "po_id":       r['po_id'],
                    "amount":      float(r['total_amount']),
                    "limit":       float(r['approval_amount_limit']),
                    "approved_by": r['approved_by'],
                    "description": f"审批人 {r['approved_by']} 权限 "
```

```
                                f"¥{r['approval_amount_limit']:,.2f}, "
                                f"但审批了 ¥{r['total_amount']:,.2f}",
                    "ontology_ref": "proc:ExceedsApprovalAuthority"
                })

    # ── 规则 3：供应商黑名单 ──────────────────────────────────
    if not rule_types or "BLACKLISTED_VENDOR" in rule_types:
        async with pool.acquire() as conn:
            rows = await conn.fetch("""
                SELECT po.po_id, po.vendor_id, v.vendor_name, po.total_amount
                FROM purchase_orders po
                JOIN vendors v ON po.vendor_id = v.vendor_id
                WHERE v.blacklisted = TRUE
                  AND po.status != 'PO_CANCELLED'
                  AND ($1::varchar[] IS NULL OR po.po_id = ANY($1))
            """, po_ids)
            for r in rows:
                violations.append({
                    "rule_id":     "PROC-RULE-003",
                    "rule_type":   "BLACKLISTED_VENDOR",
                    "severity":    "CRITICAL",
                    "po_id":       r['po_id'],
                    "vendor_id":   r['vendor_id'],
                    "vendor_name": r['vendor_name'],
                    "description": f"向黑名单供应商 {r['vendor_name']} 下达采购订单",
                    "ontology_ref": "proc:BlacklistedVendorTransaction"
                })

    high_count   = sum(1 for v in violations if v['severity'] == 'HIGH')
    crit_count   = sum(1 for v in violations if v['severity'] == 'CRITICAL')

    return {
        "is_compliant":   len(violations) == 0,
        "total_issues":   len(violations),
        "critical_count": crit_count,
        "high_count":     high_count,
        "violations":     violations
    }
```

7. 端到端场景演示：三路匹配异常分析

7.1 场景描述

用户请求：「分析 2025 年 Q1 采购业务的三路匹配情况，识别所有异常发票并给出处理建议」

以下展示系统完整的执行轨迹，包含每一步的输入输出：

7.2 完整调用链路

```
# ── 端到端执行示例 ──────────────────────────────────────────

import asyncio, json
from datetime import date

async def run_demo():
    # Step 1: 用户请求进入 Orchestrator
    user_request = "分析 2025 年 Q1 采购业务的三路匹配情况，识别异常发票并给出处理建议"
    print("=" * 60)
    print(f"用户请求: {user_request}")
    print("=" * 60)

    # Step 2: 意图解析结果（模拟）
    intent = {
        "scenario":     "THREE_WAY_MATCH",
        "time_range":   {"start": "2025-01-01", "end": "2025-03-31"},
        "filters":      {"company_code": "1000"},
        "analysis_depth": "DRILLDOWN",
        "required_agents": ["p2p_agent", "rule_agent", "report_agent"],
        "priority_kpis": ["三路匹配率", "异常发票金额", "合规违规数"]
    }
    print(f"
[Orchestrator] 意图解析结果:
"
          f"{json.dumps(intent, ensure_ascii=False, indent=2)}")

    # Step 3: DAG 执行（模拟真实耗时）
    print("
[DAG] 开始并行执行...")
    print("  并行层 1: fetch_pos(0.8s) | fetch_gr(0.6s) | "
          "fetch_invoices(0.9s) | load_ontology(0.3s)")
    print("  → 实际耗时: 0.9s (最慢任务)")
    print("  并行层 2: three_way_match(4.2s) | rule_validation(2.1s)")
    print("  → 实际耗时: 4.2s")
    print("  串行层 3: generate_report(3.8s)")
    print("  → 总耗时: ~9s（串行需 ~13s）")

    # Step 4: 模拟三路匹配结果
    match_result = {
        "summary": {
            "total_pos":      245,
            "matched":        231,
            "mismatched":     14,
            "match_rate_pct": 94.29,   # 低于 98%基准，触发预警
            "total_variance": 387420.50
        },
        "mismatches": [
            {
                "po_id": "PO-2025-00831",
                "vendor_name": "上海精密零件有限公司",
                "po_amount":   485000.00,
                "gr_amount":   482000.00,
```

```
                "inv_amount":  510000.00,
                "variance_pct": 5.81,    # 超过 5%阈值
                "match_status": "MISMATCH"
            },
            {
                "po_id": "PO-2025-00756",
                "vendor_name": "广州电子配件厂",
                "po_amount":   128000.00,
                "gr_amount":   125600.00,
                "inv_amount":  128000.00,   # 发票与 PO 匹配但收货短缺
                "variance_pct": 1.91,
                "match_status": "MISMATCH"
            }
        ]
    }

    # Step 5: 合规校验结果
    compliance_result = {
        "is_compliant": False,
        "total_issues": 3,
        "violations": [
            {
                "rule_type":   "NO_PR_BACKING",
                "severity":    "HIGH",
                "po_id":       "PO-2025-00901",
                "amount":      56800.00,
                "description": "PO-2025-00901 金额¥56,800 无对应采购申请"
            }
        ]
    }

    # Step 6: 最终报告（由 Report Agent 生成）
    final_report = {
        "summary": "2025 年 Q1 三路匹配率 94.29%，低于 98%行业基准。"
                   "14 张发票存在匹配异常，总差异金额¥387,420.50；"
                   "同时发现 3 起合规违规，含 1 起越权采购。",

        "findings": [
            {
                "title":       "三路匹配率低于行业基准",
                "severity":    "HIGH",
                "description": "Q1 三路匹配率 94.29%，低于 98%基准 4.71 个百分点。"
                               "14 张异常发票集中在 3 家供应商。",
                "kpi_ref":     "proc:ThreeWayMatchRate",
                "recommendation": "对 3 家高频异常供应商启动专项约谈，"
                                  "要求其规范发票开具流程"
            },
            {
                "title":       "PO-2025-00831 发票金额异常偏高",
                "severity":    "HIGH",
                "description": "发票金额¥510,000，比收货单¥482,000 多¥28,000"
                               "（+5.81%，超过 5%阈值）。建议冻结付款。",
                "kpi_ref":     "proc:ThreeWayMatchRule",
                "recommendation": "向供应商发送差异确认函，要求提供价格变动说明"
            }
        ],

        "kpis": {
            "three_way_match_rate":  "94.29%  ⚠️ 低于 98%基准",
            "total_variance_amount": "¥387,420.50",
            "compliance_issues":     3,
            "blocked_invoice_count": 14
        },
```

```
        "action_items": [
            {
                "priority": 1,
                "action":   "冻结 PO-2025-00831 对应发票付款，联系供应商核实",
                "owner":    "采购主管",
                "deadline": "2025-04-03"
            },
            {
                "priority": 2,
                "action":   "对无 PR 的 PO-2025-00901 启动合规调查",
                "owner":    "内控部门",
                "deadline": "2025-04-05"
            },
            {
                "priority": 3,
                "action":   "对 3 家高频异常供应商开展约谈，制定改善计划",
                "owner":    "供应商管理团队",
                "deadline": "2025-04-10"
            }
        ]
    }

    print("
" + "=" * 60)
    print("最终分析报告: ")
    print(json.dumps(final_report, ensure_ascii=False, indent=2))
    print("=" * 60)
    return final_report

# 执行
asyncio.run(run_demo())
```

7.3 关键设计决策汇总

设计点	选择方案	原因 / 避免的问题
工具调用安全	参数化查询 + 意图白名单	防止 Agent 产生恶意 SQL 导致数据泄露
数值计算	专用计算引擎，不用 LLM	LLM 浮点计算不可靠，财务数字容不得误差
本体上下文	按模块过滤注入，非全量	避免无关知识污染，节省 70% context token
写操作控制	Human-in-the-loop 审批	ERP 写操作有真实业务后果，不能全自动
并行调度	DAG 拓扑排序驱动	不让 LLM 实时判断串并行，结果不可复现
三路匹配	SQL 计算 + 规则引擎	确定性校验，结果可审计，不依赖 LLM 判断
状态持久化	Redis 检查点，不信任内存	避免 API 超时导致长任务从头重跑

7.4 扩展：其他采购分析场景

本架构可直接扩展至以下场景，复用 Orchestrator + 工具层，只需增加对应 DAG 和 Sub-Agent system prompt：

分析场景	核心 KPI	主要工具
供应商绩效季报	准时交货率、质量合格率、PPV	get_vendor_scorecard + calculate_ppv
年度采购成本优化	品类 PPV、合同覆盖率	calculate_ppv + spend_analysis
付款到期管理	到期发票金额、早付折扣机会	query_invoices + payment_forecast
采购周期时间分析	PR→PO 平均天数、审批积压	po_cycle_time + approval_bottleneck
物料短缺风险预警	库存水位、在途量、需求预测	stock_level + mrp_simulation

— 文档完 —